


CENTRO TECNOLÓGICO

CURSO DE CIÊNCIAS DA COMPUTAÇÃO

(BACHARELADO)

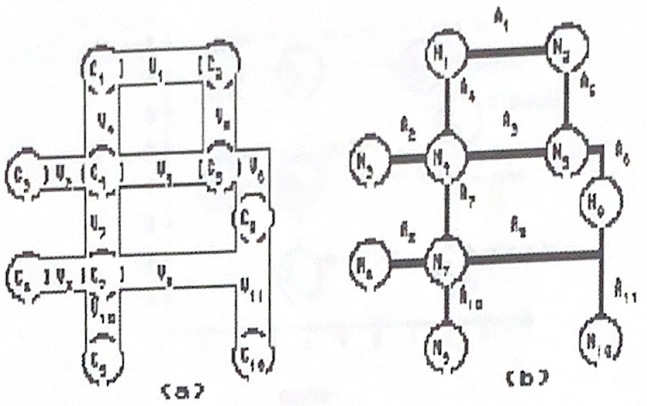
# UMA INTERFACE GRÁFICA PARA O SISTEMA DE CRUZAMENTO

RELATÓRIO DO TRABALHO DE CONCLUSÃO DE CURSO SUBMETIDO A UNIVERSIDADE REGIONAL DE BLUMENAU PARA OBTENÇÃO DOS CRÉDITOS DE DISCIPLINA COM NOME EQUIVALENTE NO CURSO DE CIÊNCIAS DA COMPUTAÇÃO - BACHARELADO

DALTON SOLANO DOS REIS

BLUMENAU, DEZEMBRO/1992.

92/2-03

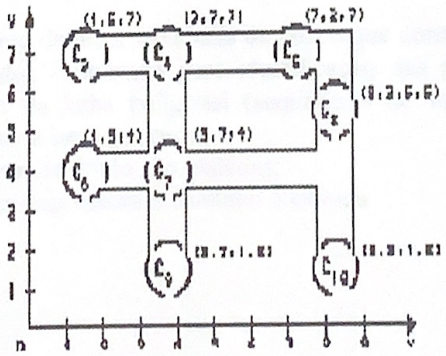


onde

**Ci** : i-esimo cruzamento
**Vj** : j-esima via
**Nk** : k-esimo nodo
**Ap** : p-esimo arco

**Figura 2.1 - malha viária vs (b) estrutura de grafo**

Se à malha viária acrescentarmos um sistema de referência, pode-se associar, a cada cruzamento, um par de coordenadas (Figura 2.2). O caminho que liga dois cruzamentos quaisquer pode, então, ser interpretado como uma linha poligonal (seqüência de segmentos), que une os dois cruzamentos cujos vértices têm associados a si pares de coordenadas (Figura 2.3).



onde

**(x,y)** representa as coordenadas
do centro do cruzamento

**Figura 2.2 - malha viária com sistema de referência**

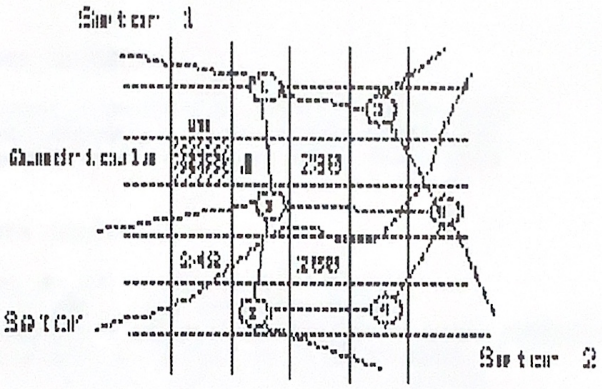


**Figura 2.4 - setor e cruzamento nas quadrículas**

Por sua vez, a determinação da linha poligonal que representa o caminho a ser visualizado realiza-se em duas etapas:

i. identificação de todos os pares de nodos presentes no caminho;
ii. para cada par de nodos identificados no item anterior, obter os vários segmentos de reta que os unem.

De modo a implementar este processo, foram definidas as seguintes estruturas de dados:

```
estrutura NODO
{
   setor : tipo_setor;
   cruzamento : tipo_cruzamento;
}

estrutura ARCO
{
   nodo_origem : tipo estrutura NODO;
   nodo_destino : tipo estrutura NODO;
   código_segmento_inicial : tipo_código_segmento;
   número_de_segmentos : tipo_número_segmentos;
}

estrutura VÉRTICE
{
   coordenada_x : inteiro;
   coordenada_y : inteiro;
}

estrutura SEGMENTO
{
   código_segmento : tipo_código_segmento;
   vértice_inicial : tipo estrutura VÉRTICE;
   vértice_final : tipo estrutura VÉRTICE;
```

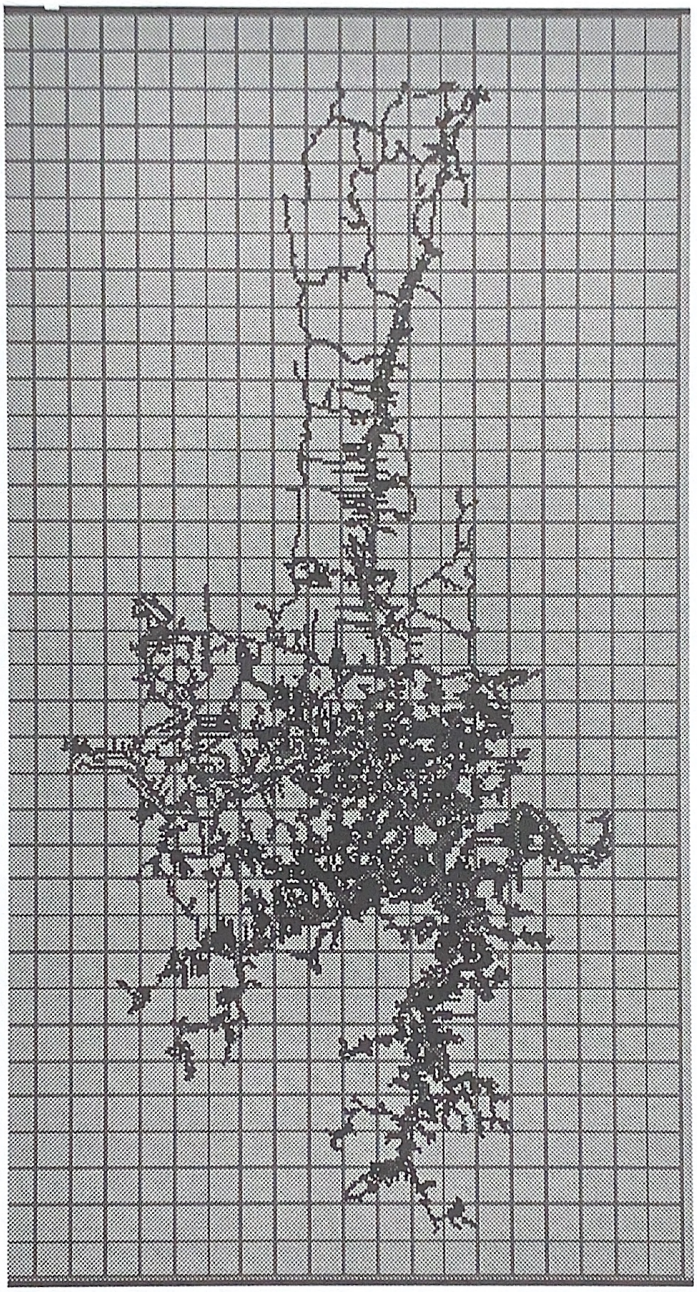


**Figura 3.1 - mapa viário com três articulações**

A utilização da coordenada_x_base e coordenada_y_base permite definir valores de coordenadas (para o vértice_inicial, vértice_final e pontos de referência) em relação às coordenadas do vértice inferior esquerdo da quadrícula. Isto permite suas representações com números da ordem $10^2$, ao invés de $10^4$, possibilitando uma economia média de oito dígitos (bytes) por segmento (arquivo QUA-###.VIA, QUA-###.RIO, QUA-###.REF) o que representa uma redução média de 33,55% no tamanho dos arquivos. O uso de arquivos tipados possibilitou uma redução ainda maior. Na média, a economia foi de 73,53% em relação aos tamanhos originais (Tabela 3.5).

| Arquivo | Número segmentos | Tamanho (bytes) sem coordenada_x_base e coordenada_y_base | Resultados obtidos com uso de coordenadas base | | Resultados obtidos com uso de arquivo tipado | |
|---|---|---|---|---|---|---|
| | | | tam. (bytes) | economia | tam. (bytes) | economia |
| 375-VIA | 1 | 48 | 33 | 31,25 % | 8 | 83,33 % |
| 254-VIA | 405 | 10.511 | 6.669 | 36,55 % | 3.232 | 69,25 % |
| 174-VIA | 797 | 19.907 | 13.370 | 32,84 % | 6.368 | 68,01 % |

**Tabela 3.1- economia de espaço obtido com o uso de coordenadas base**

O programa utilizado para calibração, mapeamento, encaixe das quadrículas, captura das coordenadas dos pontos e geração dos arquivos acima, encontra-se listado no Apêndice A-03.

## 3.7 CALIBRAÇÃO

Antes de iniciar a digitalização de uma quadrícula torna-se necessária a realização de uma calibração, onde se faz coincidir as coordenadas de seus vértices na mesa digitalizadora com as dos vértices de um retângulo exibido no vídeo. Após efetua-se o mapeamento das coordenadas da mesa para as coordenadas da quadrícula na grade.

- **botões do caminho:** encontram-se a direita da janela de mensagens e são utilizados para movimentar o cursor do caminho entre os diversos cruzamentos;
- **botões da vista geral:** dispostos sob a janela da vista geral, no total de oito botões. Os quatro superiores servem para movimentar o cursor da grade. Os inferiores com as opções AMPLIA e REDUZ (a serem implementados), MARCA e OK. A opção MARCA serve para marcar/desmarcar quadrículas na grade, enquanto OK permite a visualização da área selecionada na janela de exibição.

A disposição dos elementos da Interface Gráfica observou como princípios de projeto, alinhamento, balanceamento e as proporções entre suas várias áreas [Foley-90].

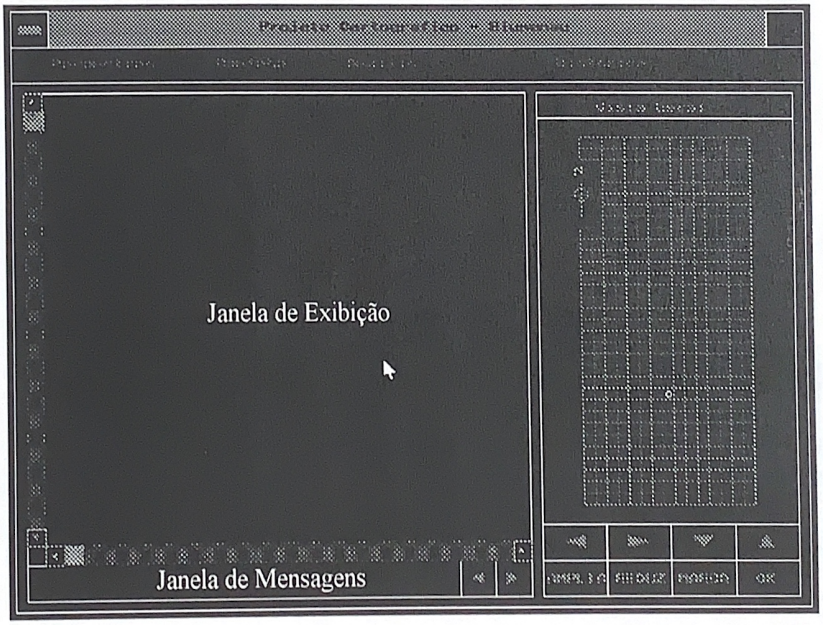



**Figura 5.1 - interface do programa**

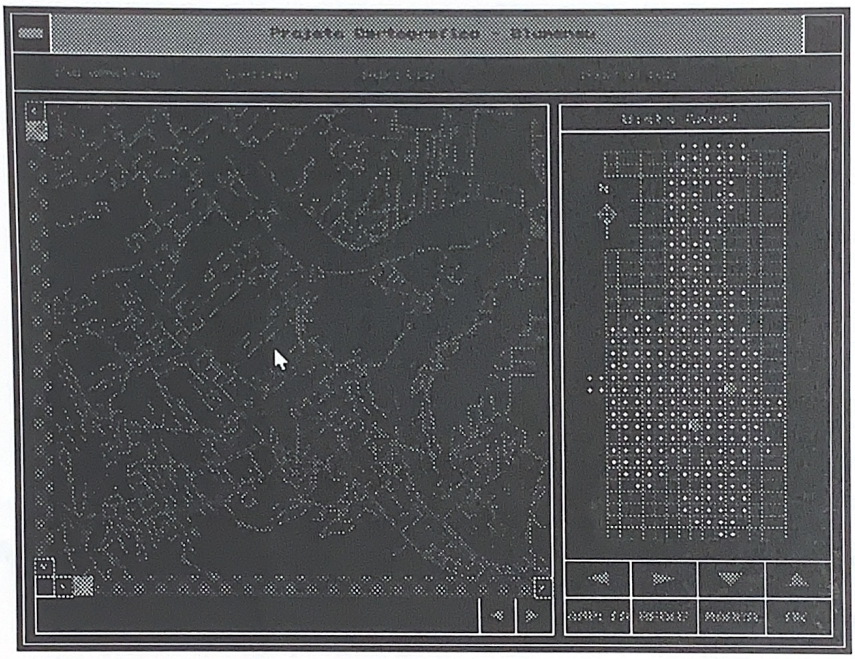


**Figura 5.4 - Seleção de área para exibição**

## 5.3 ESTRUTURAÇÃO DO PROGRAMA

O programa em seu atual estágio de implementação, se compõem de 7 módulos básicos e seus relacionamentos se encontram representados na Figura 5.5. A Figura 5.7 apresenta um modelo, em análise orientada a objetos, utilizando durante o projeto do Software.

```
     (command "zoom" "A")
;             -- Configura o tipo de ambiente de trabalho.
     (command "blipmode" "OFF")  ; desliga os blips.
     (command "ucsicon"  "OFF")  ; desliga o icon de coordenadas.
     (command "snap"     "OFF")  ; desliga o snap.
;      ** falta ligar as coordenadas.
   )
;
;
;-------------------------------------------------------------------
;   Cria Layer GRADE e constroi a grade de quadriculas.
;         -> cor  : vermelho (red)
;         -> linha: continua (continuous)
;
   (defun Grade()
;             -- Define um novo Layer GRADE.
     (command "layer" "M" "GRADE" "C" "RED" "GRADE" "L" "CONTINUOUS"
            "GRADE" nil)
;              -- Perimetro externo das Quadriculas.
     (command "line" '(0.0 0.0) '(20000.0 0.0) '(20000.0 35000.0)
                     '(0.0 35000.0) '(0.0 0.0) nil)
;              -- Constroi as Colunas das Quadriculas.
     (setq Colunas 19)
     (setq Contador 0)
     (setq Incremento 0.0)
     (while (< Contador Colunas)
       (setq Contador (+ Contador 1))
       (setq Incremento (* 1000.0 Contador))
       (setq PontoIni (cons Incremento '(0.0)))
       (setq PontoFin (cons Incremento '(35000.0)))
       (command "line" PontoIni PontoFin nil)
     )
;              -- Constroi as Linhas das Quadriculas.
     (setq Linhas  34)
     (setq Contador 0)
     (setq Incremento 0.0)
     (while (< Contador Linhas)
       (setq Contador (+ Contador 1))
       (setq Incremento (* 1000.0 Contador))
       (setq Incremento (cons Incremento '()))
       (setq PontoIni (cons '0.0     Incremento))
       (setq PontoFin (cons '20000.0 Incremento))
       (command "line" PontoIni PontoFin nil)
     )
;              -- Define Layer 0  e CONGELA o Layer GRADE.
     (command "layer" "S" "0" "F" "GRADE" nil)
   )
;
;
;-------------------------------------------------------------------
;   Numera as quadriculas da Grade de 1 a 700.
;         -> cor  : verde (green)
;         -> linha: continua (continuous)
;
   (defun NumeraGrade()
;             -- Define um novo Layer NumeraGrade.
     (command "layer" "M" "NUMERAGRADE" "C" "GREEN" "NUMERAGRADE"
                      "L" "CONTINUOUS" "NUMERAGRADE"  nil)
;             -- Loop para numeracao das quadriculas.
     (setq CONTADOR 0.0)
```